
Concepts of Programming Language

— Introduction —

Prof. Dr. Fazlul Hasan Siddiqui

Office: DUET | **Education:** BSc: IUT; MSc: BUET; PhD: ANU

Credit: Thomas Wies | Book: M. L. Scott

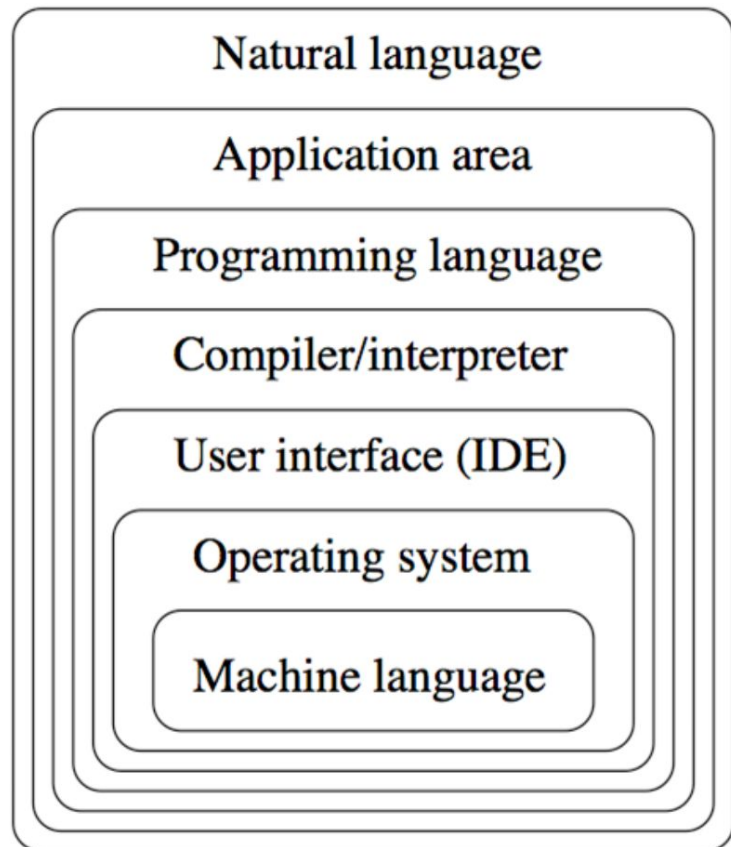
Reference books

- R. W. **Sebesta**.
 - Concepts of Programming Languages
- M. L. **Scott**.
 - Programming language pragmatics
- J. C. **Mitchell**.
 - Concepts in programming languages
- T. W. **Pratt** and M. V. Zelkowitz.
 - Programming Languages: Design and implementation
- R. **Harper**.
 - Practical Foundations for Programming Languages
- Some programming-language ‘family t`rees’ (too big for slide):
 - <http://www.levenez.com/lang/>
 - <http://rigaux.org/language-study/diagram.html>

What is a (Programming) Language?

A language is a vocabulary and set of grammatical rules for communication between people.

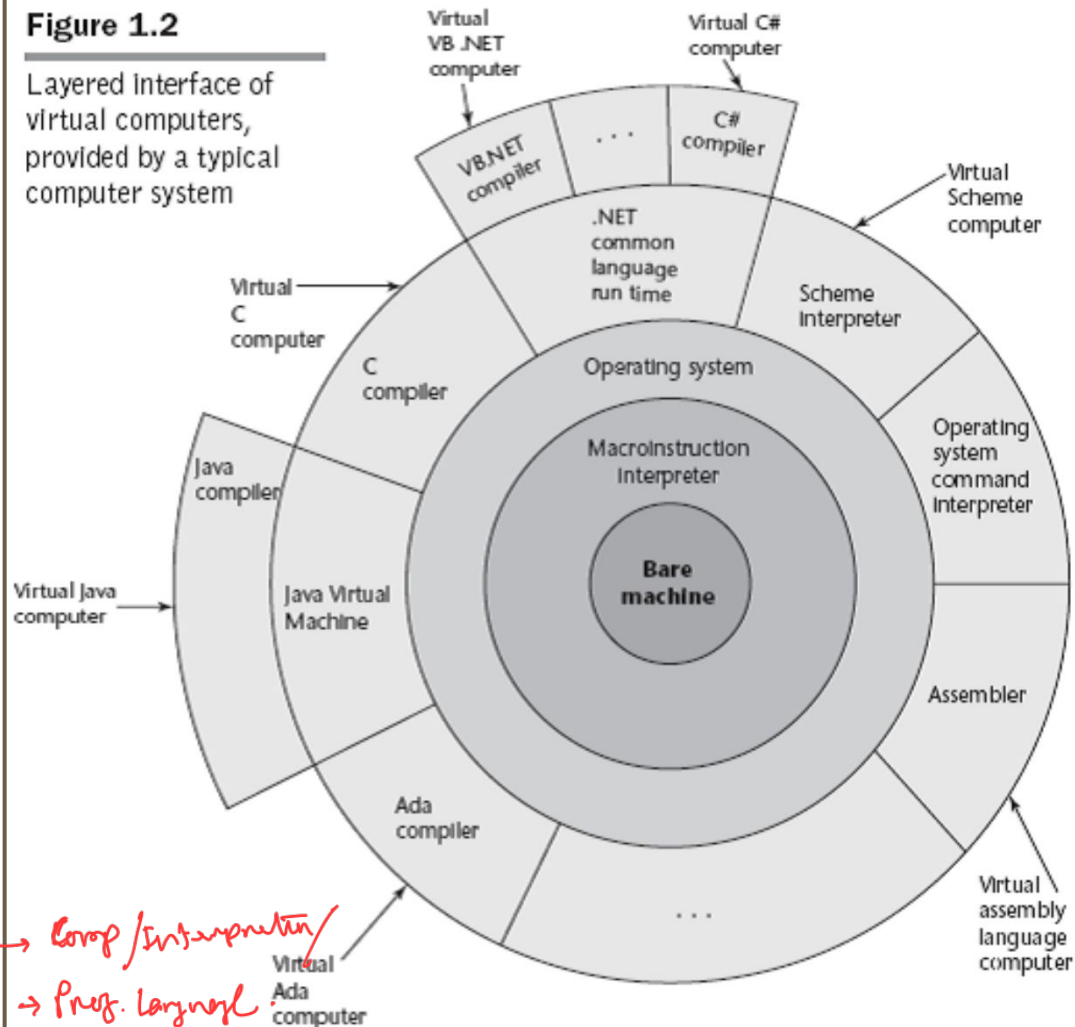
A programming language is a vocabulary and set of grammatical rules for instructing a computer to perform specific tasks. (Definition from webopedia)



H/W → P/W → O/S → VM → App → IDE → Comp/Interpreter
 → Prog. Language

Figure 1.2

Layered interface of virtual computers, provided by a typical computer system



Programming Abstractions

what are they?

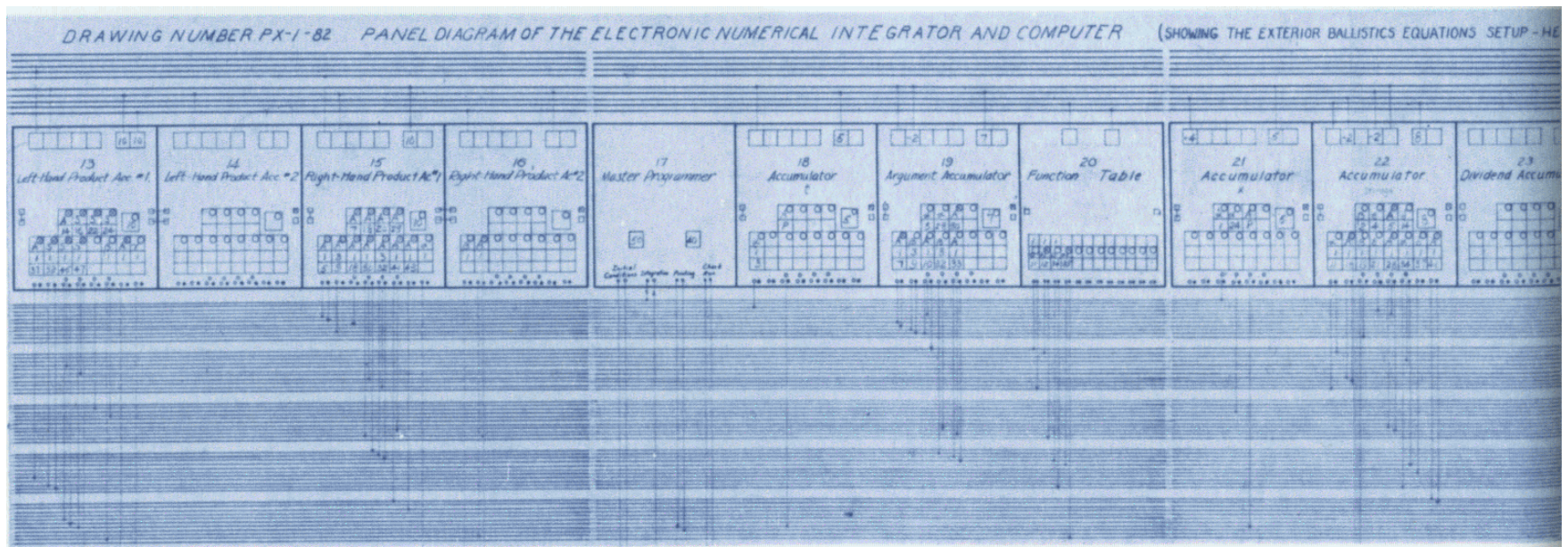
constructs that hide their implementation

↑
"abstract away"

↑
"boilerplate",
"plumbing"

ENIAC (1946, University of Philadelphia)

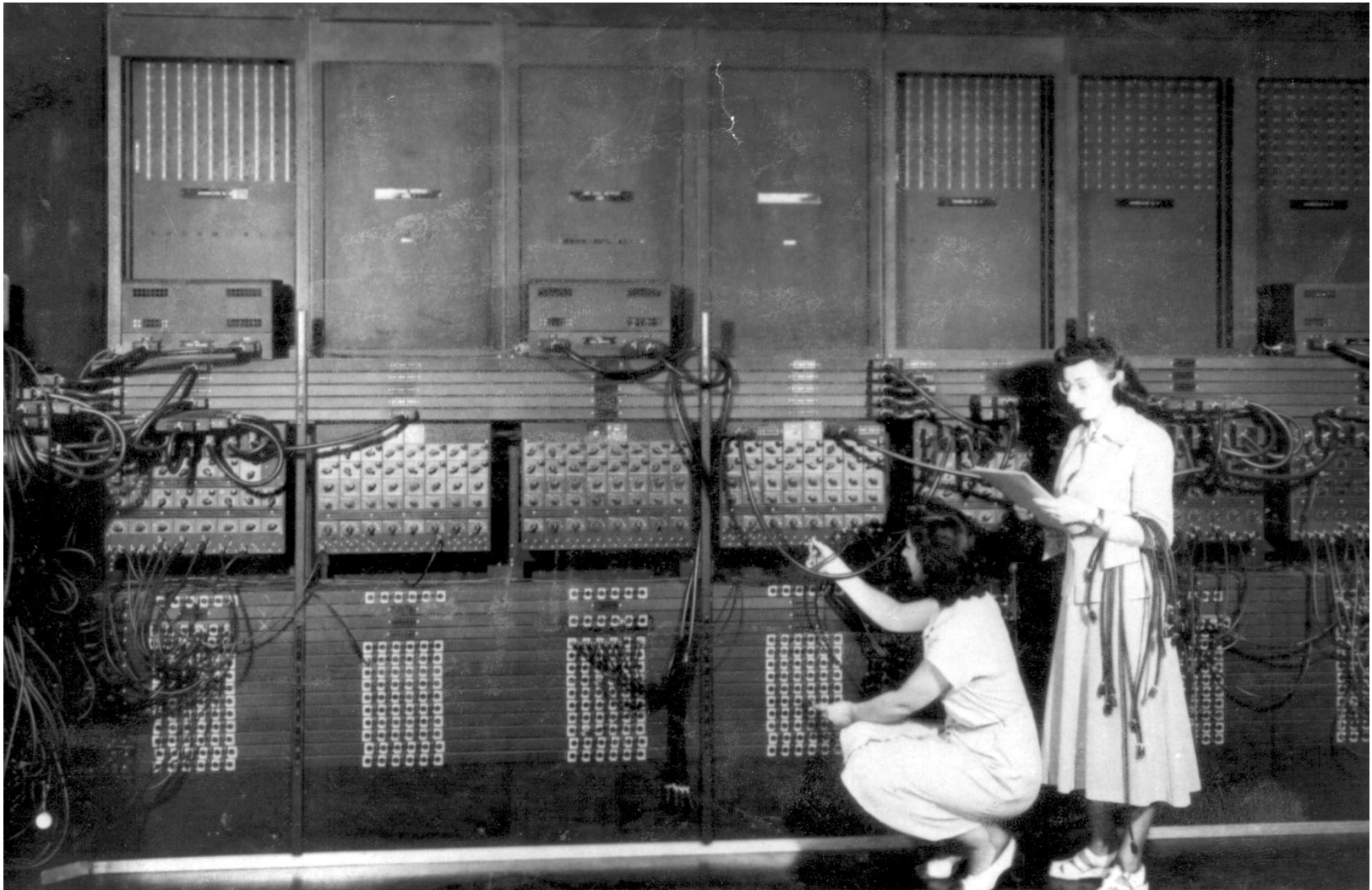
ENIAC program for external ballistic equations:



- > ENIAC's first actual use was running a program for external ballistics — calculating artillery shell trajectories.
- > The program solved differential equations of motion for projectiles, accounting for variables like muzzle velocity, air drag, gravity, wind, and shell weight to produce firing tables — reference charts soldiers used to aim artillery accurately.

Programming the ENIAC

- > A single program could require hundreds of cable connections and switch settings.
- > Reprogramming meant physically rewiring the entire machine, which took days. There was no software — the wiring was the program.



ENIAC (1946, University of Philadelphia)

programming done by

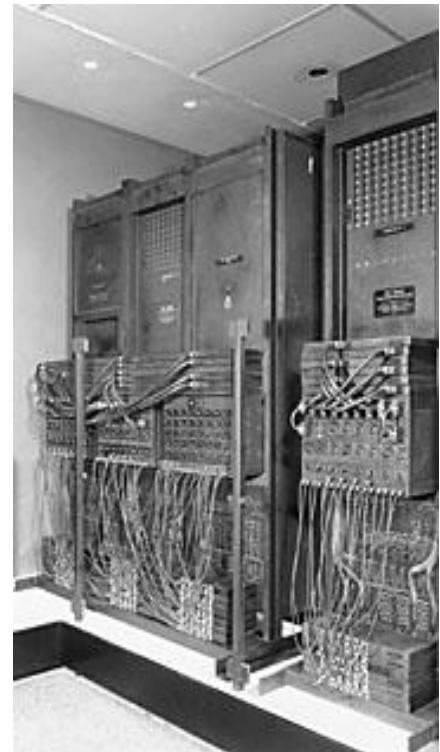
- rewiring the interconnections
- to set up desired formulas, etc

Problem (what's the tedious part?)

- programming = rewiring
- slow, error-prone

solution:

- **store the program in memory!**
- birth of *von Neuman* paradigm



- 30 ton
- 1800 sq ft
- 5000 instructions/sec
- world's 1st military anti-very fast table

UDSAC (1947, Cambridge University)

/manchester.

the first real computer

- large-scale, fully functional, **stored-program** electronic digital computer (by Maurice Wilkes) *(william tube for main memory.)*

problem: Wilkes realized:

- “a good part of the remainder of my life was going to be spent in finding errors in ... programs”

solution: procedures (1951)

- procedure: abstracts away the implementation
- reusable software was born

Assembly – the language (UNIVAC 1, 1950)

Idea: mnemonic (assembly) code

- Then translate it to machine code by hand (no compiler yet)
- write programs with mnemonic codes (add, sub),
with symbolic labels,
- then assign addresses by hand

Example of symbolic assembler

clear-and-add a

add b

store c

translate it by hand to something like this (understood by CPU)

B100 A200

C300

Assembler – the compiler (Manchester, 1952)

- it was assembler nearly as we know it, called AutoCode
- a loop example, in MIPS, a modern-day assembly code:

> MIPS (Microprocessor without Interlocked Pipeline Stages) is a RISC (Reduced Instruction Set Computer) assembly language with simple, fixed-size 32-bit instructions.

loop: `addi $t3, $t0, -8`

`addi $t4, $t0, -4`

`lw $t1, theArray($t3)`

`lw $t2, theArray($t4)`

`add $t5, $t1, $t2`

`sw $t5, theArray($t0)`

`addi $t0, $t0, 4`

`blt $t0, 160, loop`

`jr $ra`

Gets the last

two elements

Adds them together...

...and stores the result

Moves to next "element"

of theArray

If not past the end of

theArray, repeat

ouch, why do labels need to be visible to programmers?

Assembly programming caught on, but

Problem: Software costs exceeded hardware costs!

↳ updates

John Backus: “Speedcoding” *(high level language.)*

- An interpreter for a high-level language
- Ran 10-20 times slower than hand-written assembly
 - way too slow

← need compiler optimizations

FORTRAN I (1954-57)

Language, and the first compiler

- Produced code almost as good as hand-written
- Huge impact on computer science (laid foundations for cs164)
- Modern compilers preserve its outlines
- FORTRAN (the language) still in use today

By 1958, >50% of all software is in FORTRAN

Cut development time dramatically

- 2 weeks → 2 hrs
- that's more than 100-fold
- newer version is still popular for numerical & scientific computing efficiency. (e.g. linear algebra)
- ODP from 2003 versions.
- legacy & community support.

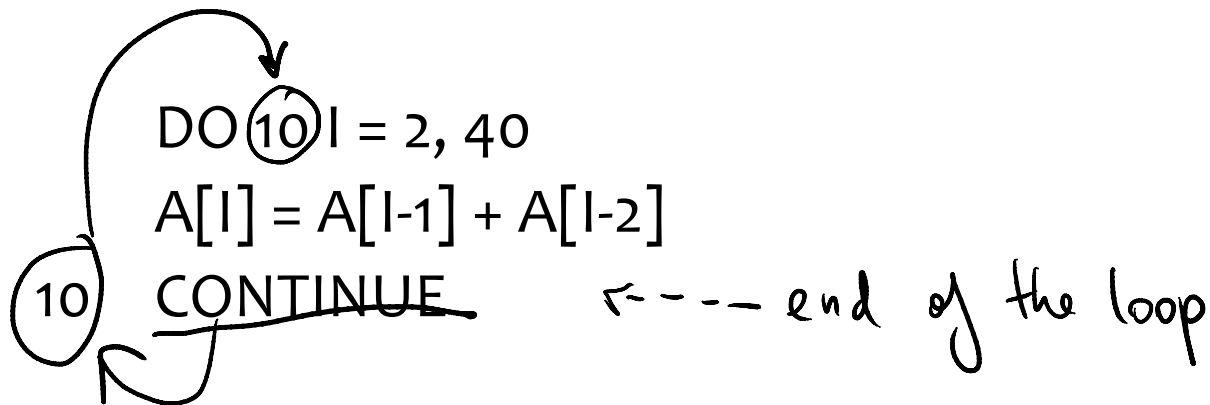
FORTRAN I (IBM, John Backus, 1954)

Example: nested loops in FORTRAN

- a big improvement over assembler,
- but annoying artifacts of assembly remain:
 - labels and rather explicit jumps (CONTINUE)
 - lexical columns: the statement must start in column 7

- The MIPS loop from previous slide, in FORTRAN:

column 1~5 $\Rightarrow$ line #
column 6 $\Rightarrow$ contd-
char. (+/")



Side note: designing a good language is hard

Good language protects against bugs, but lessons take a while.
An example that caused a failure of a NASA planetary probe:

buggy line:

```
DO 15 I = 1.100
```

what was intended (a dot had replaced the comma):

```
DO 15 I = 1,100
```

because Fortran ignores spaces, compiler read this as:

```
DO15I = 1.100
```

which is an assignment into a variable DO15I, not a loop.

This mistake is harder to make (if at all possible) with the modern lexical rules (white space not ignored) and loop syntax

```
for (i=1; i < 100; i++) { ... }
```

Goto considered harmful

L1: statement

if expression goto L1
statement

Dijkstra says: gotos are harmful

- use structured programming
- lose some performance, gain a lot of readability

how do you rewrite the above code into structured form?

Object-oriented programming (1970s)

The need to express that more than one object supports draw() :

```
draw(2DElement p) {  
    switch (p.type) {  
        SQUARE: ... // draw a square  
                break;  
        CIRCLE: ... // draw a circle  
                break;  
    }  
}
```

Problem:

unrelated code (drawing of SQUARE and CIRCLE) mixed in same procedure

Solution:

Object-oriented programming with inheritance

Object-oriented programming

In Java, the same code has the desired separation:

```
class Circle extends 2DElement {  
    void draw() { <draw circle> }  
}  
class Square extends 2DElement {  
    void draw() { <draw circle> }  
}
```

the dispatch is now much simpler:

```
p.draw()
```

Review of historic development

- wired interconnects → stored program (von Neuman machines)
- lots of common bugs in repeated code → procedures
- machine code → symbolic assembly (compiled by hand)
- tedious compilation → assembler (the assembly compiler)
- assembly → FORTRAN I
- gotos → structured programming
- hardcoded “OO” programming → inheritance, virtual calls

Do you see a trend?

- Removal of boilerplate code
 - also called **plumbing**, meaning it conveys no application **logic**
- Done by means of new abstractions, such as procedures
 - They abstract away from details we don't wish to reason about

Where will languages go from here?

The trend is towards higher-level abstractions

- express the algorithm concisely!
- which means hiding often repeated code fragments
- new constructs hide more of these low-level details.

Haven't we abstracted most programming plumbing?

- we might have
- it is likely that new abstractions will be mostly domain specific ==> small language will prevail

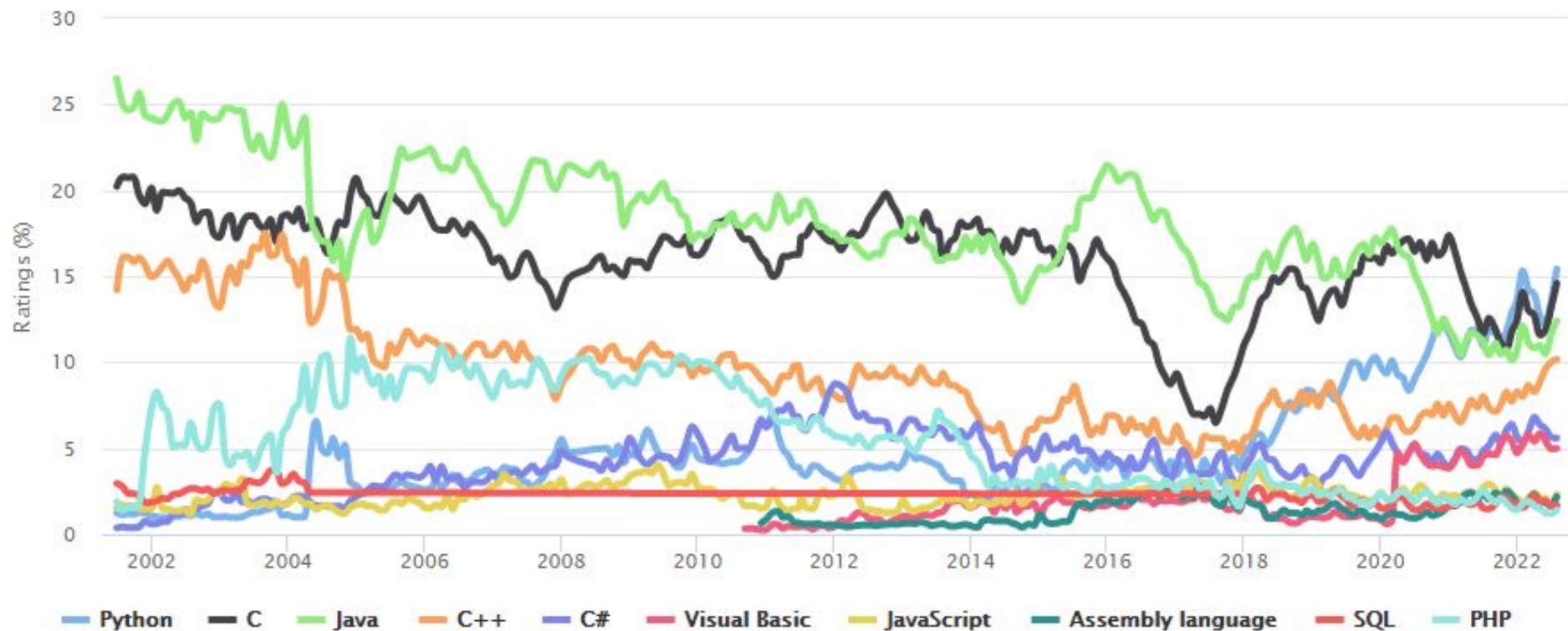
TIOBE Programming Language of the Year award 2023

Jan 2023	Jan 2022	Change	Programming Language	Ratings	Change
1	1		 Python	16.36%	+2.78%
2	2		 C	16.26%	+3.82%
3	4	▲	 C++	12.91%	+4.62%
4	3	▼	 Java	12.21%	+1.55%
5	5		 C#	5.73%	+0.05%
6	6		 Visual Basic	4.64%	-0.10%
7	7		 JavaScript	2.87%	+0.78%
8	9	▲	 SQL	2.50%	+0.70%
9	8	▼	 Assembly language	1.60%	-0.25%
10	11	▲	 PHP	1.39%	-0.00%
11	10	▼	 Swift	1.20%	-0.21%
12	13	▲	 Go	1.14%	+0.10%
13	12	▼	 R	1.04%	-0.21%
14	15	▲	 Classic Visual Basic	0.98%	+0.01%
15	16	▲	 MATLAB	0.91%	-0.05%
16	18	▲	 Ruby	0.80%	-0.08%
17	14	▼	 Delphi/Object Pascal	0.73%	-0.27%
18	26	▲	 Rust	0.61%	+0.11%
19	20	▲	 Perl	0.59%	-0.12%
20	23	▲	 Scratch	0.58%	-0.01%

<https://tiobe.com/tiobe-index/>

TIOBE Programming Community Index

Source: www.tiobe.com



New Languages will Keep Coming

Why are there so many programming languages?

- **Evolution**
 - Structured → Object Oriented → Functional → Logic
- **Special purposes**
 - Lisp for symbols; Snobol for strings; C for systems; etc.
- **Expressive power**
 - No one language is good at expressing all programming styles
- **Personal preference**
 - Programmers have their own personal tastes
- **What makes languages evolve?**
 - Changes in hardware or implementation platform
 - Changes in attitudes to safety and risk
 - New ideas from academic or industry

What makes a language successful?

- Expressive power.
- Ease of use for the novice.
- Ease of implementation.
- Standardisation & Orthogonality
- Many useful libraries.
- Excellent compilers (including open-source)
- Programming environments.
- Portability of programs.
- Economics, patronage, and inertia. Note the recent trend of big companies to create/control their own languages: C# (**Microsoft**), Hack (**Facebook**), Go (**Google**), Objective-C/Swift (**Apple**), Rust (**Mozilla**) and perhaps even Python (**Dropbox** hired Guido van Rossum).

Why Study Programming Languages?

- One or two languages is not enough for a computer scientist.
- You should know
 - the general concepts beneath the requirements
 - choices in designing programming languages.

Be prepared to program in new languages

Languages undergo constant change

- FORTRAN 1953
- ALGOL 60 1960
- C 1973
- C++ 1985
- Java 1995

Evolution steps: 12 years per widely adopted language

- are we overdue for the next big one?

... or is the language already here?

- *Hint:* are we going through a major shift in what computation programs need to express?
- your answer here:

JavaScript

Python

· programs are distributed
· more interactive
· "installed" as part of web page

Better understanding of significance of implementation

- The best programmers are the ones having at least understanding of **how things work under the hood**
 - Understand the implementation issues
- You can simply write a code and let the compiler do everything, but knowing implementation details helps you to **use a language more intelligently** and **write the code that is more efficient**
- Also, it allows you to visualize how a computer executes language constructs
 - Cost optimization; e.g. recursion is slow

Better use of languages that are already known

- Many programming languages are large and complex
 - It is uncommon to know all the features
- By studying the concepts of programming languages, programmers can learn about previously unknown parts of the languages easily.

Overall advancement of computing

- New ways of thinking about computing, new technology, hence need for new appropriate language concepts
- Not to repeat history
 - Although ALGOL 60 was a better language (with better block structure, recursion, etc) than FORTRAN, it did not become popular. If those who choose languages are better informed, better languages would be more popular.

Develop your own language

Are you kidding? No. Guess who developed:

- PHP
- Ruby
- JavaScript
- Perl

Done by smart hackers like you

- in a garage
- not in academic ivory tower

Our goal: learn good academic lessons

- so that your future languages avoid known mistakes

Ability to Design New Languages

- You may need to design a special purpose language to enter the commands for a software that you develop.
 - A language for an robotics interface
- Studying PL concepts will give you the ability to design a new language suitable and efficient for your software requirements.

Increased Ability to Express Ideas

- Natural languages:
 - The depth at which people think is influenced by the expressive power of the language.
 - The more appropriate constructs you have, the easier it is to communicate.
 - It is difficult for people to conceptualize structures that they cannot describe verbally.

Increased Ability to Express Ideas

- Programming Languages:
 - This is similar for PL's. The language in which you develop software puts limits on the kinds of data structures, control structures and abstractions that can be used.
 - Awareness of a wider variety of programming language features can reduce such limitations in software development.
 - Programmers increase the range of software development by learning new language constructs.
 - For example, if you learn associate arrays in Perl, you can simulate them in C.

Improved background for choosing appropriate languages

- Not every programming language can be suitable for all the software requirements.
- Many programmers learn one or two languages specific to the projects.
- Some of those languages may no longer be used.
- When they start a new project they continue to use those languages which are old and not suited to the current projects.

Improved background for choosing appropriate languages

- However another language may be more appropriate to the nature of the project.
 - Lots of text processing -> Perl may be a good option.
 - Lots of matrix processing -> MATLAB can be used.
- If you are familiar with the other languages, you can choose better languages.
- Studying the principles of PLs provides a way to judge languages:
 - “The advantages of Perl for this problem are.....”, “The advantages of Java are....”

Increased ability to learn new languages

- Programming languages are still evolving
 - Many languages are very new, new languages can be added in time.
-  If you know the programming language concepts you will learn other languages much easier.
 - For example, if you know concept of object oriented programming, it is easier to learn C++ after learning Java
- Just like natural languages,
 - The better you know the grammar of a language, the easier you find to learn a second language.
 - learning new languages actually causes you to learn things about the languages you already know